NativeWind: React Native + Tailwind

Diego Muñoz

09 de noviembre de 2026

React Native + Tailwind

- Plantilla con **NativeWind** ya configurado.
- Código en estilo **JavaScript**, sin tipos.
- Conceptos básicos de **diseño móvil**: layout, scroll, safe area.

Punto de partida

- La plantilla oficial evita configurar todo a mano:

```
pnpx rn-new <nombre-del-proyecto> --nativewind --pnpm
cd <nombre-del-proyecto>
pnpm add expo@^58.0.0
pnpm expo install --fix
pnpm start
```

- El proyecto viene con:
  - Expo
  - NativeWind
  - Tailwind v3
  - Metro + Babel ya configurados

  Aunque los archivos sean `.tsx`, el código se escribe como si fuera JS.

Estructura base

Archivos importantes que ya vienen listos:

- `App.tsx`: punto de entrada.
- `components/`: un archivo por ejemplo. Tailwind ya lee esta carpeta.
- `tailwind.config.js`: config de Tailwind + NativeWind.
- `babel.config.js`: plugin de NativeWind.
- `metro.config.js`: integra `global.css`.
- `global.css`: importa `@tailwind base`, `components`, `utilities`.
  No es necesario tocar la configuración, solo el código de la app.

NativeWind

- Biblioteca para usar **clases tipo Tailwind** en React Native.
- Traduce `bg-blue-500`, `text-xl`, `flex-1`, etc. a estilos nativos.
- Funciona sobre Expo con el setup generado por `rn-new`.

Primer test con NativeWind

Crear components/PrimerTest.tsx. className reemplaza a style:

```tsx
import { Text, View } from "react-native";

export default function PrimerTest() {
  return (
    <View className="flex-1 items-center justify-center bg-slate-900">
      <View className="px-6 py-4 rounded-2xl bg-slate-800">
        <Text className="text-2xl font-semibold text-white">React</Text>
        <Text className="text-base text-slate-300 mt-2">Native</Text>
      </View>
    </View>
  );
}
```

App.tsx

`App.tsx` importa `global.css` y renderiza el ejemplo:

```tsx
import "./global.css";
import { SafeAreaProvider } from "react-native-safe-area-context";
import PrimerTest from "./components/PrimerTest";

export default function App() {
  return (
    <SafeAreaProvider>
      <PrimerTest />
    </SafeAreaProvider>
  );
}
```

Levantar el proyecto

- Se abre el panel de Expo Dev Tools.
- Escanear el código QR con **Expo Go**.
- Si algo se rompe tras editar, hacer **Reload** desde la app o reiniciar `pnpm start`.

Web vs móvil

- Pantalla pequeña y orientación variable.
- No hay **hover**, solo toques/gestos.
- Texto debe ser legible en pantallas con DPI alto.
- Considerar **notch**, barra de estado y zonas no utilizables.
- Usuario muchas veces en movimiento.

Flexbox en React Native

- **Flexbox**: ordena los hijos de un contenedor en fila o en columna.
- En React Native todo `View` es flex, en columna por defecto.
  - `flex-1`: ocupa todo el espacio disponible.
  - `flex-row`: pone los hijos en fila.
  - `justify-between`: reparte los hijos a lo largo de la fila, con el espacio entre ellos.
  - `items-center`: centra los hijos en vertical.
- Crear `components/Flexbox.tsx`. En `App.tsx`, importar `Flexbox` y cambiar `<PrimerTest />` por `<Flexbox />`.

components/Flexbox.tsx

```tsx
import { Text, View } from "react-native";

export default function Flexbox() {
  return (
    <View className="flex-1 flex-row items-center justify-between px-4">
      <Text>Left</Text>
      <Text>Right</Text>
    </View>
  );
}
```

Scroll básico

- `View` no hace scroll. El contenido que no cabe queda cortado.
- `ScrollView` permite desplazarlo, en vertical por defecto.
- `"Texto largo. ".repeat(200)` genera texto suficiente para ver el efecto.
- Crear `components/Scroll.tsx`. En `App.tsx`, importar `Scroll` y cambiar `<Flexbox />` por `<Scroll />`.

components/Scroll.tsx

```tsx
import { ScrollView, Text, View } from "react-native";

export default function Scroll() {
  return (
    <ScrollView className="flex-1 bg-slate-900">
      <View className="p-6">
        <Text className="text-2xl font-bold text-white mb-4">About</Text>
        <Text className="text-base text-slate-200">
          {"Texto largo. ".repeat(200)}
        </Text>
      </View>
    </ScrollView>
  );
}
```

Safe area

- Zonas ocupadas por notch, barra de estado, barra de gestos.
- Evitar que el contenido quede “debajo” de esas áreas.
- `SafeAreaView` agrega el espacio necesario para esquivarlas.
- `SafeAreaProvider`, en `App.tsx`, mide esas zonas.
- Crear `components/SafeArea.tsx`. En `App.tsx`, importar `SafeArea` y cambiar `<Scroll />` por `<SafeArea />`.

components/SafeArea.tsx

El rojo es el espacio que reserva `SafeAreaView`:

```tsx
import { SafeAreaView } from "react-native-safe-area-context";
import { Text } from "react-native";

export default function SafeArea() {
  return (
    <SafeAreaView className="flex-1 bg-red-500">
      <Text className="flex-1 bg-slate-900 text-white">Contenido</Text>
    </SafeAreaView>
  );
}
```

Toque en vez de click

- `Pressable` recibe toques. `onPress` se ejecuta en cada toque.
- `useState` guarda la cantidad de toques.
- Crear `components/Contador.tsx`. En `App.tsx`, importar `Contador` y cambiar `<SafeArea />` por `<Contador />`.

components/Contador.tsx

```tsx
import { useState } from "react";
import { Pressable, Text, View } from "react-native";
export default function Contador() {
  const [count, setCount] = useState(0);
  return (
    <View className="flex-1 items-center justify-center bg-slate-900">
      <Pressable
        className="px-6 py-3 rounded-full bg-emerald-500"
        onPress={() => setCount(count + 1)}>
        <Text className="text-white font-semibold">Toques: {count}</Text>
      </Pressable>
    </View>
  );
}
```

Navegación entre ejemplos

- Una barra abajo cambia de ejemplo sin editar `App.tsx`.
- `useState` guarda el índice del ejemplo activo.
- `key={index}` reinicia el ejemplo al cambiar, el contador vuelve a 0.
- El `App.tsx` final reemplaza al anterior.

Imports de App.tsx

```tsx
import "./global.css";
import { useState } from "react";
import { Pressable, Text, View } from "react-native";
import { SafeAreaProvider } from "react-native-safe-area-context";
import PrimerTest from "./components/PrimerTest";
import Flexbox from "./components/Flexbox";
import Scroll from "./components/Scroll";
import SafeArea from "./components/SafeArea";
import Contador from "./components/Contador";
```

Lista de ejemplos

```
const examples = [
  { label: "Primer test", Component: PrimerTest },
  { label: "Flexbox", Component: Flexbox },
  { label: "Scroll", Component: Scroll },
  { label: "Safe area", Component: SafeArea },
  { label: "Contador", Component: Contador },
];
```

Pantalla activa

```
export default function App() {
  const [index, setIndex] = useState(0);
  const { Component } = examples[index];
  return (
    <SafeAreaProvider>
      <View className="flex-1">
        <Component key={index} />
      </View>
```

Barra de navegación

```
      <View className="flex-row flex-wrap bg-yellow-300 pb-8">
        {examples.map(({ label }, i) => (
          <Pressable
            key={label}
            onPress={() => setIndex(i)}
            className={i === index ? "p-3 bg-yellow-500" : "p-3"}
          >
            <Text className="font-bold">{label}</Text>
          </Pressable>
        ))}
      </View>
    </SafeAreaProvider>
  );
}
```

Resumen

- `pnpx rn-new --nativewind --pnpm` genera un proyecto Expo con:
  - NativeWind
  - Tailwind v3
  - Babel + Metro configurados
- Código estilo **JS** aunque los archivos sean `.tsx`.
- Diseño móvil:
  - Pantalla pequeña y DPI alto.
  - Flexbox.
  - `ScrollView`.
  - `SafeAreaView`.
  - Componentes táctiles con `Pressable`.

Preguntas y Discusión

¿Tienes dudas? ¡Hablemos!